

РЕФЕРАТ

Пояснительная записка 30 с., 8 рис., 1 табл., 8 источн.

НЕЙРОСЕТЕВОЙ ПОИСК, СЕМАНТИЧЕСКИЙ ПОИСК, ЯЗЫКОВЫЕ МОДЕЛИ, EMBEDDINGS, SENTENCE TRANSFORMERS, TMDB.

Объект проектирования – веб-интерфейс для семантического поиска фильмов, реализуемый с помощью фреймворка *Streamlit* (локальное веб-приложение).

Цель проекта – разработать систему нейросетевого поиска фильмов по смысловым описаниям сюжета с использованием языковых моделей.

В процессе работы над проектом:

1. Проведен анализ датасета «*TMDB 5000 Movies*» и определена его пригодность для задачи.
2. Изучены возможности библиотеки *Sentence Transformers* и выбрана мультилингвальная модель для генерации эмбеддингов.
3. Реализован механизм семантического поиска на основе косинусного сходства векторных представлений.
4. Разработан интерактивный веб-интерфейс с помощью *Streamlit* для ввода запросов и отображения результатов.

В дальнейшем планируется пользовательское тестирование, чтобы собрать обратную связь от потенциальных пользователей, улучшить точность поиска (например, за счёт *fine-tuning* модели) и добавить дополнительные функции, такие как рекомендации на основе истории запросов.

В результате был разработано локальное веб-приложение «*MovieSense*» на базе *Streamlit*, которое позволяет выполнять нейросетевой поиск фильмов по естественному языковому описанию сюжета (на русском и английском языках), с поддержкой фильтрации по годам выпуска и отображением степени семантической схожести результатов.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 Постановка задачи.....	7
2 Изучение теоретических основ для разработки программного решения	9
2.1 Теория.....	9
2.2 Инструментарий для проектного решения.....	10
3 Разработка решения	13
3.1 Разработка веб-приложения	13
3.2 Версионный контроль.....	17
3.3 Демонстрация работы веб-приложения.....	17
3.4 Оценка эффективности нейросетевого поиска	19
4 Индивидуальный вклад участника	22
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	26
ПРИЛОЖЕНИЕ А	27

ВВЕДЕНИЕ

В условиях стремительного роста цифровых медиаресурсов и постоянно расширяющейся базы фильмов пользователи всё чаще сталкиваются с трудностями при поиске нужного контента. Традиционные системы поиска, основанные на точном совпадении ключевых слов, не способны улавливать смысл запроса, сформулированного естественным языком. Например, фраза «фильм про путешествие в будущее с неожиданной развязкой» не содержит конкретных названий, жанров или имён актёров, но несёт чёткое смысловое описание, которое современные каталоги часто игнорируют.

Актуальность проекта обусловлена необходимостью перехода от ключевого поиска к семантическому (нейросетевому) подходу, способному понимать намерения пользователя на основе контекста, а не только по формальным признакам. Такой подход позволяет находить релевантные фильмы даже при абстрактной, эмоциональной или неточной формулировке запроса, значительно повышая удобство и точность поиска.

Целью данного проекта является разработка локального веб-приложения «*MovieSense*», реализующего нейросетевой поиск фильмов по смысловому описанию сюжета с использованием языковых моделей.

Для достижения поставленных целей необходимо решить несколько ключевых задач:

- 1) Провести анализ датасета «*TMDB 5000 Movies*» и определить пригодность его структуры для задачи семантического поиска.
- 2) Исследовать возможности библиотеки *Sentence Transformers* и выбрать подходящую языковую модель для генерации эмбедингов текста.
- 3) Реализовать механизм преобразования описаний фильмов и пользовательских запросов в единое семантическое пространство и организовать поиск на основе косинусного сходства.

- 4) Разработать интерактивный веб-интерфейс с помощью фреймворка *Streamlit*, обеспечивающий удобный ввод запросов и наглядное отображение результатов с указанием степени семантической схожести.

Таким образом, реализация данного проекта позволит пользователям находить фильмы не по ключевым словам, а по смыслу их описания – даже если запрос сформулирован неформально, эмоционально или без указания конкретных названий, жанров или актёров. Это упростит поиск нужного контента, сделает взаимодействие с каталогом фильмов более интуитивным и приблизит пользовательский опыт к естественному диалогу.

1 Постановка задачи

Проектная идея

Проект посвящён разработке локального веб-приложения «*MovieSense*», предназначенного для интеллектуального поиска фильмов по смысловому описанию сюжета. Основная идея заключается в том, чтобы пользователь мог находить нужные фильмы, описывая их своими словами – например: «фильм про дружбу двух роботов на заброшенной планете» или «мелодрама с несчастной любовью и дождём в финале». Приложение использует современные языковые модели для понимания смысла запроса и сопоставления его с описаниями из базы данных, обеспечивая поиск, основанный не на ключевых словах, а на семантическом сходстве.

Проблема

Современные каталоги фильмов (такие как КиноПоиск, *IMDb* и другие) ориентированы на структурированный поиск: по названию, году, жанру или актёрам. Однако пользователи часто не помнят точных деталей фильма, но хорошо помнят его атмосферу, сюжетные повороты или эмоции, которые он вызвал. Традиционные системы не способны интерпретировать такие запросы, что приводит к неудовлетворённости пользователей и трудностям при поиске. Отсутствие инструментов, понимающих естественный язык на смысловом уровне, создаёт барьер между человеком и нужным контентом.

Цель

Цель проекта – создать локальное веб-приложение «*MovieSense*», реализующее нейросетевой (семантический) поиск фильмов по свободному текстовому описанию. Решение должно позволять находить релевантные фильмы на основе смысла запроса, поддерживать запросы на русском и английском языках, отображать степень схожести результатов и

предоставлять возможность фильтрации (например, по году выпуска). Это сделает поиск фильмов более интуитивным, естественным и эффективным.

2 Изучение теоретических основ для разработки программного решения

2.1 Теория

Для успешной реализации проекта «*MovieSense*» была проведена теоретическая подготовка, охватывающая ключевые направления, лежащие в основе семантического поиска и нейросетевых технологий:

1. Нейросетевой (семантический) поиск информации

Исследованы современные подходы к *Neural Information Retrieval*, в которых релевантность определяется не по совпадению ключевых слов, а по смысловой близости между запросом и документом. Также были рассмотрены принципы преобразования текста в плотное векторное пространство и методы измерения схожести, такие как косинусное расстояние.

2. Языковые модели и векторные представления текста

Проанализированы архитектуры предобученных языковых моделей, способных генерировать эмбединги для предложений. Особое внимание уделено моделям из семейства *Sentence-BERT*, оптимизированным именно для задач семантического сходства, включая мультилингвальные версии, поддерживающие русский и английский языки.

3. Работа с эмбедингами и векторными базами данных

Изучены подходы к хранению и быстрому поиску векторных представлений. Рассмотрены инструменты, такие как *FAISS* и *ChromaDB*, позволяющие эффективно выполнять поиск ближайших соседей в пространстве высокой размерности, а также добавлять к векторам метаданные (название фильма, год выпуска и т.д.).

4. Обработка и подготовка текстовых данных

Исследованы методы предварительной обработки текстов: очистка, нормализация и фильтрация. Уделено внимание вопросам качества исходных описаний в датасетах и их влиянию на точность поиска.

5. Разработка веб-интерфейсов для ИИ-приложений

Проанализированы современные подходы к созданию интерфейсов, ориентированных на взаимодействие с семантическими моделями. Рассмотрены принципы проектирования, обеспечивающие ясное отображение результатов поиска и обратной связи (например, визуализация процента схожести).

6. Оценка качества семантического поиска

Изучены метрики, используемые для оценки релевантности результатов: *Hit@K*, *Mean Reciprocal Rank*, а также практики субъективной оценки на тестовых запросах.

Полученные теоретические знания легли в основу архитектурных и технологических решений при разработке «*MovieSense*». Они позволили обоснованно выбрать инструменты, оптимизировать процесс векторизации и обеспечить корректную интерпретацию пользовательских запросов на естественном языке.

2.2 Инструментарий для проектного решения

Для реализации проекта «*MovieSense*» используется набор современных инструментов с открытым исходным кодом, ориентированных на разработку ИИ-приложений и веб-интерфейсов. Все компоненты выбраны с учётом задачи семантического поиска, простоты развёртывания и возможности локального запуска:

1. *Python* – основной язык программирования, объединяющий все компоненты проекта. С его помощью реализованы обработка текстовых данных, генерация эмбеддингов, работа с векторной

базой данных и логика веб-интерфейса. Управление зависимостями осуществляется через виртуальные окружения и систему установки пакетов *pip*, что обеспечивает изоляцию проекта и воспроизводимость среды разработки.

2. *Sentence Transformers* – библиотека для работы с предобученными языковыми моделями, оптимизированными под задачи семантического сравнения предложений. Используется для преобразования описаний фильмов и пользовательских запросов в векторные представления.
3. *ChromaDB* – лёгкая встраиваемая векторная база данных, позволяющая хранить эмбединги и связанные метаданные (название фильма, год выпуска и т.д.), а также выполнять быстрый поиск по косинусному сходству без необходимости внешнего сервера.
4. *Streamlit* – фреймворк для быстрой разработки интерактивных веб-интерфейсов на *Python*. Обеспечивает создание локального веб-приложения с минимальным количеством кода, поддерживая ввод текста, отображение результатов и фильтрацию.
5. Датасет «*TMDB 5000 Movies*» – открытый датасет, содержащий информацию о более чем 5000 фильмах, включая краткие сюжетные описания на английском языке. Используется как источник данных для построения векторного индекса.
6. *GitHub* – платформа для хостинга и совместной разработки программного обеспечения с использованием системы контроля версий *Git*. Обеспечивает централизованное хранение исходного кода, отслеживание изменений, организацию командной работы и управление версиями проекта с поддержкой *pull-request*, код-ревью и автоматизированных проверок.

Данный инструментарий позволяет реализовать полный цикл разработки: от подготовки данных и создания поискового движка до развёртывания удобного локального веб-интерфейса без зависимости от облачных сервисов.

3 Разработка решения

3.1 Разработка веб-приложения

На начальном этапе разработки «*MovieSense*» был проведен комплексный анализ датасета «*TMDB 5000 Movies*». Датасет содержит сведения о более чем 5 тысячах фильмов, включая краткие сюжетные описания. Проведённый анализ подтвердил пригодность структуры данных для семантического поиска: поле «*overview*» предоставляет достаточно подробные текстовые описания, подходящие для векторизации. В ходе предобработки были удалены записи с отсутствующими или слишком короткими описаниями, а также извлечён год выпуска. Результат предобработки представлен на рисунке 1.

	title	overview	release_date	runtime	year
0	Avatar	In the 22nd century, a paraplegic Marine is di...	2009-12-10	162.0	2009
1	Pirates of the Caribbean: At World's End	Captain Barbossa, long believed to be dead, ha...	2007-05-19	169.0	2007
2	Spectre	A cryptic message from Bond's past sends him o...	2015-10-26	148.0	2015
3	The Dark Knight Rises	Following the death of District Attorney Harve...	2012-07-16	165.0	2012
4	John Carter	John Carter is a war-weary, former military ca...	2012-03-07	132.0	2012
...	...	...	...	...	...
4794	El Mariachi	El Mariachi just wants to play his guitar and ...	1992-09-04	81.0	1992
4795	Newlyweds	A newlywed couple's honeymoon is upended by th...	2011-12-26	85.0	2011
4796	Signed, Sealed, Delivered	"Signed, Sealed, Delivered" introduces a dedic...	2013-10-13	120.0	2013
4797	Shanghai Calling	When ambitious New York attorney Sam is sent t...	2012-05-03	98.0	2012
4798	My Date with Drew	Ever since the second grade when he first saw ...	2005-08-05	90.0	2005

Рисунок 1 – Структура очищенного датасета с примерами записей

Основное внимание уделялось исследованию возможностей библиотеки *Sentence Transformers*. После тестирования нескольких моделей выбрана мультилингвальная модель *paraphrase-multilingual-MiniLM-L12-v2*, обеспечивающая высокую точность семантического сходства при приемлемой производительности. С помощью выбранной модели сгенерированы векторные представления всех сюжетных описаний. Полученные эмбединги

сохранены в векторной базе данных *ChromaDB* вместе с метаданными. Общая архитектура системы представлена на рисунке 2.

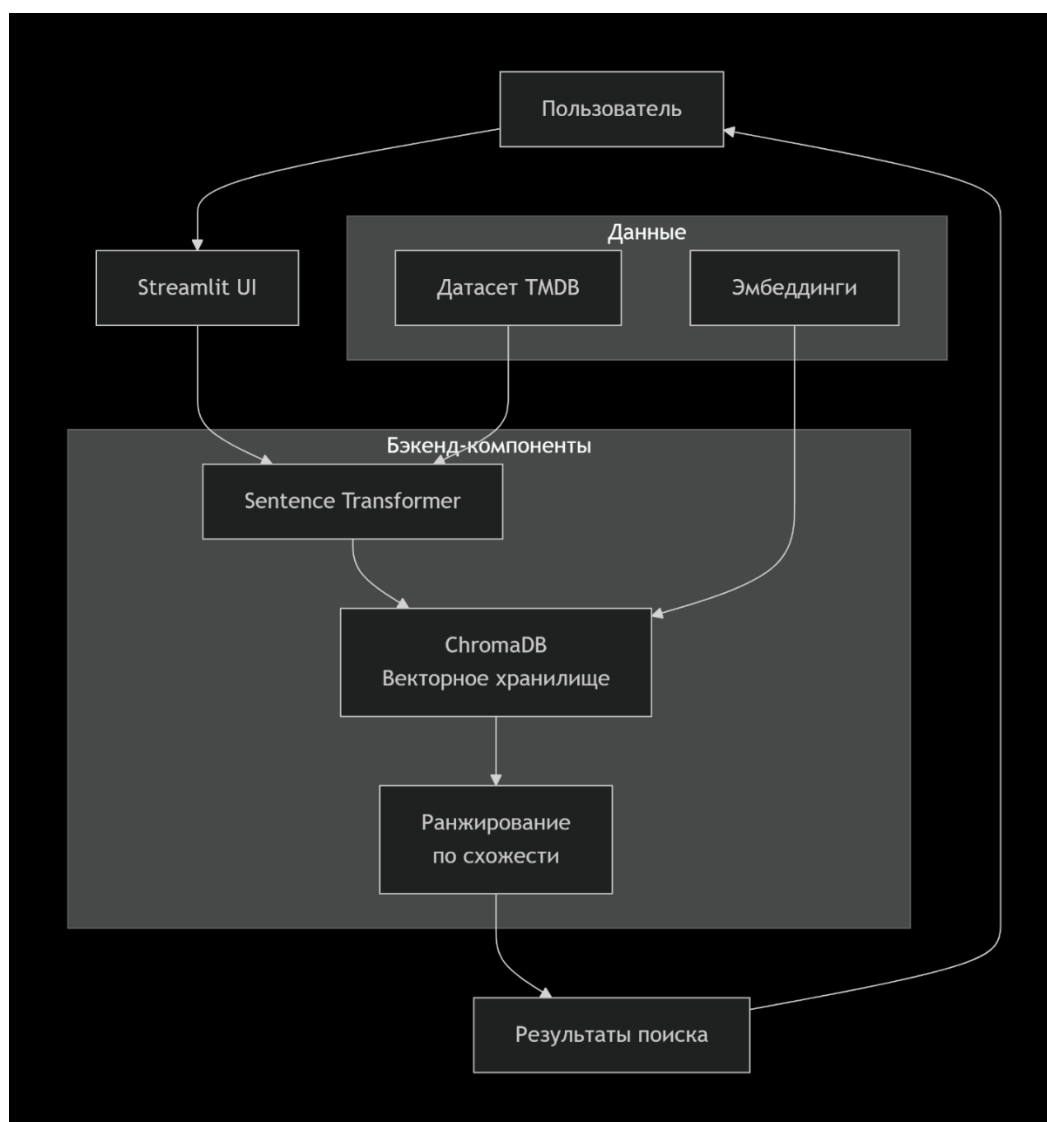


Рисунок 2 – Архитектура приложения «*MovieSense*»

После этого был реализован механизм семантического поиска, позволяющий преобразовывать как описания фильмов, так и пользовательские запросы в единое векторное пространство. Поиск осуществляется по косинусному сходству с возможностью фильтрации по годам выпуска и порогу минимальной схожести. Схема работы механизма поиска приведена на рисунке 3.

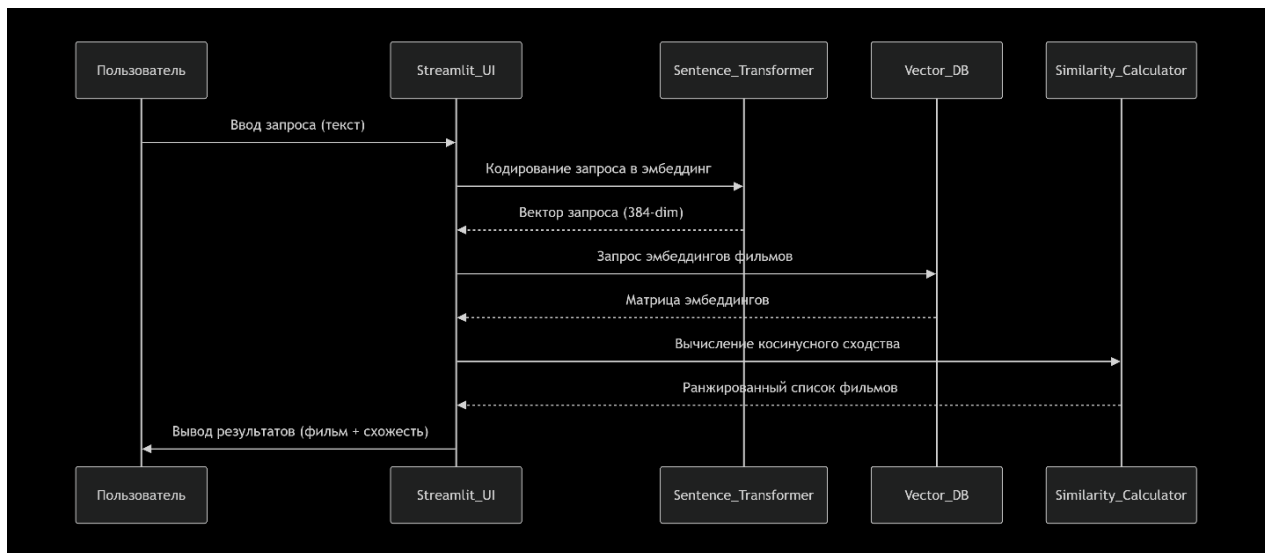


Рисунок 3 – Схема процесса семантического поиска

Алгоритм нейронного поиска реализован как последовательность шагов, обеспечивающих преобразование текстовых данных в векторное пространство и вычисление сходства. Ниже на рисунке 4 представлена блок-схема алгоритма, иллюстрирующая ключевые этапы обработки запроса.

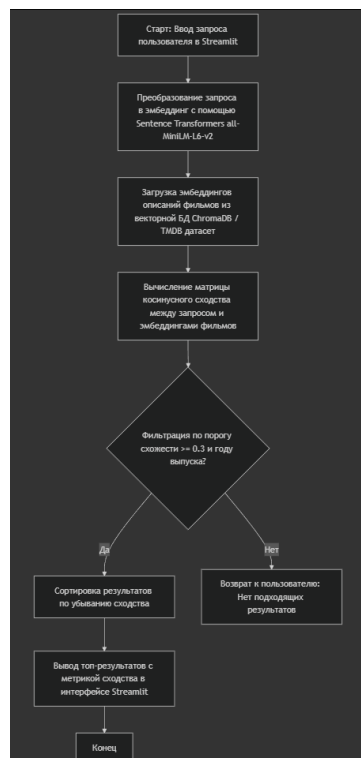


Рисунок 4 – Блок-схема алгоритма нейронного поиска

Алгоритм начинается с ввода пользовательского запроса через интерфейс *Streamlit*. Текст запроса преобразуется в векторное представление с использованием мультилингвальной модели *paraphrase-multilingual-MiniLM-L12-v2*. Аналогичные эмбединги для описаний фильмов из датасета «*TMDB 5000 Movies*» заранее вычислены и сохранены в векторной базе данных *ChromaDB*. Далее производится вычисление косинусного сходства между вектором запроса и векторами описаний фильмов. Полученные результаты фильтруются по установленному порогу минимальной схожести ($\geq 0,3$) и, при необходимости, по диапазону годов выпуска, после чего выполняется ранжирование по убыванию значения сходства и выводится список наиболее релевантных фильмов с указанием степени семантической близости.

Завершающим этапом стала разработка интерактивного веб-интерфейса с помощью фреймворка *Streamlit*. Интерфейс содержит поле для ввода запроса естественным языком, элементы фильтрации по годам и кнопку запуска поиска. Результаты отображаются в удобном виде с указанием метрики схожести. Внешний вид главной страницы «*MovieSense*» представлен на рисунке 5.

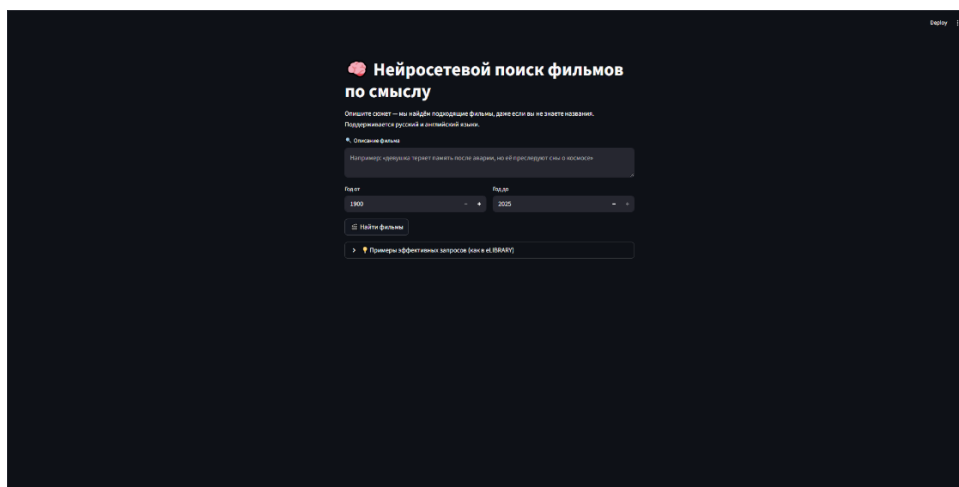


Рисунок 5 – Главная страница «*MovieSense*» с полем ввода запроса

3.2 Версионный контроль

Для координации совместной работы команды над проектом применяется система контроля версий *Git* с размещением исходного кода в репозитории на платформе *GitHub*. Основная ветка *main* защищена от прямых коммитов: все изменения вносятся исключительно через *pull request* после обязательного ревью кода участниками команды и успешного прохождения автоматизированных проверок.

Разработка нового функционала осуществляется в отдельных ветках типа *feature* с именами, отражающими содержание изменений. Это обеспечивает чистоту кодовой базы и упрощает отслеживание прогресса. Каждый коммит сопровождается подробным сообщением, содержащим тип изменения, область применения и краткое описание.

Для автоматизации рутинных операций настроены *GitHub Actions*. При каждом *push* в репозиторий автоматически запускаются процессы сборки проекта и выполнения юнит-тестов. Такой механизм позволяет своевременно обнаруживать ошибки и поддерживать высокую стабильность кода на всех этапах разработки.

Принятый подход гарантирует полную прозрачность истории изменений, способствует поддержанию высокого качества кода и значительно повышает эффективность взаимодействия всех членов команды.

Исходный код проекта, включая скрипты векторизации, веб-приложение на *Streamlit*, обработку датасета и инструкции по запуску, размещён в открытом репозитории на платформе *GitHub* – соответствующий ему *URL*-адрес указан в списке использованных источников.

3.3 Демонстрация работы веб-приложения

Веб-приложение «*MovieSense*» запускается локально и предоставляет пользователю возможность выполнять поиск фильмов по смысловому описанию сюжета.

Результат поиска по запросу «Молодой человек с уникальными способностями к математике работает уборщиком в университете и тайно решает сложнейшие математические задачи, оставленные на досках, что привлекает внимание профессоров, которые пытаются раскрыть личность таинственного гения» представлен на рисунке 6. Система успешно нашла релевантные фильмы, такие как *Good Will Hunting*, *Conspiracy Theory* и *Flubber*.

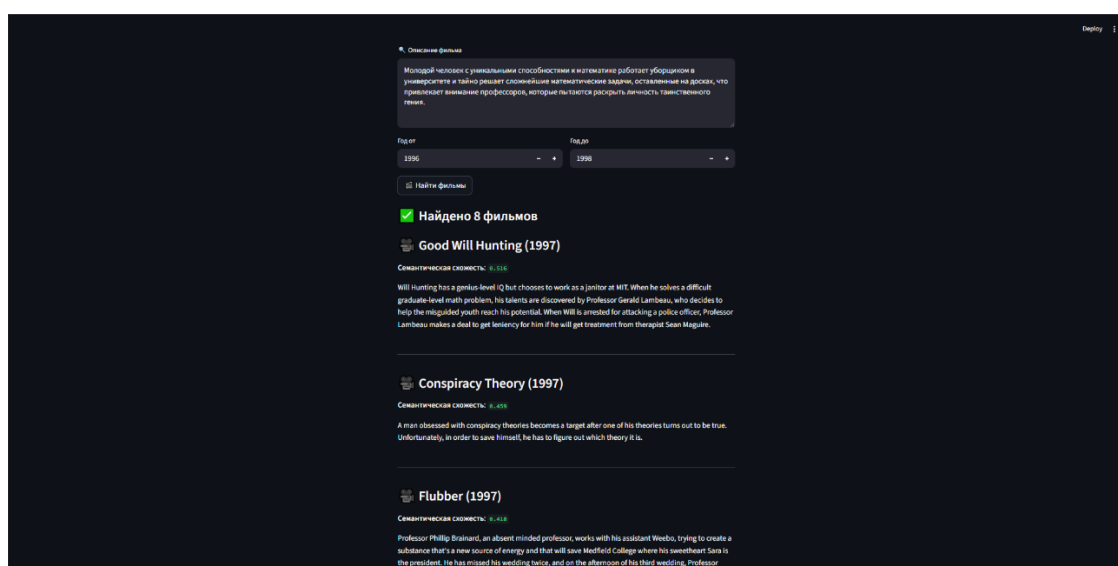


Рисунок 6 – Результаты поиска по запросу о молодом гении-математике

Пример поиска по запросу «Владелец ночного клуба в послевоенном Касабланке вынужден выбирать между любовью к женщине из прошлого и помощью её мужу, борцу за сопротивление» показан на рисунке 7. Приложение точно выявило классические картины, включая *Casablanca*, *Rebecca* и *Duel in the Sun*.

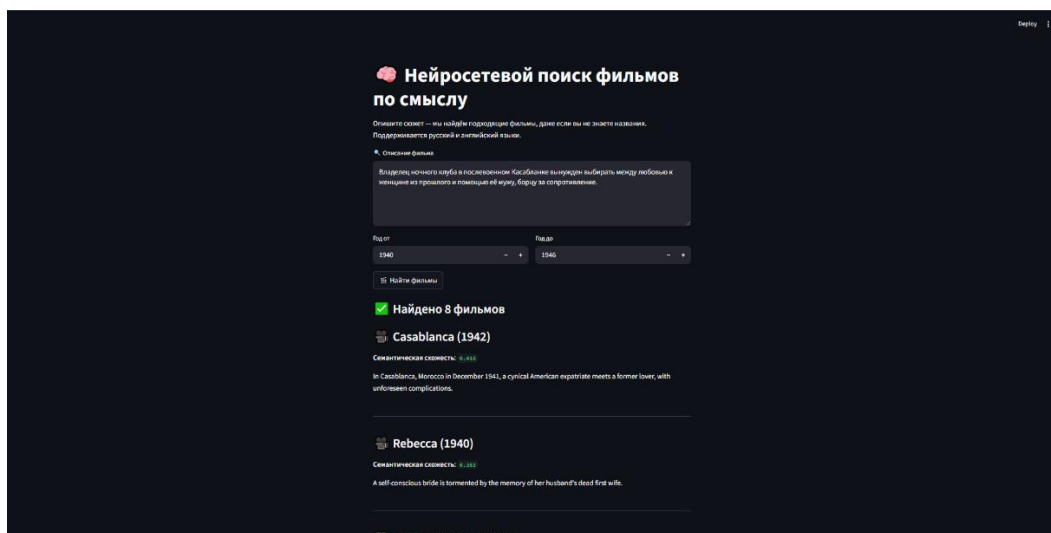


Рисунок 7 – Результаты поиска по запросу о ночном клубе в Касабланке

Демонстрация применения фильтрации по диапазону лет приведена на рисунке 6 и рисунке 7. Отображение метрики семантической схожести (в диапазоне от 0 до 1) для каждого результата также показано на этих рисунках.

3.4 Оценка эффективности нейросетевого поиска

Для объективной оценки качества работы системы семантического поиска «*MovieSense*» были использованы общепринятые в области информационного поиска метрики, адаптированные под задачу ранжирования релевантных фильмов:

1. *Precision@5* – доля релевантных фильмов среди первых 5 результатов выдачи. Эта метрика отражает полезность выдачи для пользователя, который редко смотрит дальше первой страницы.
2. *Recall@5* – доля найденных релевантных фильмов от общего числа релевантных фильмов в датасете. Метрика показывает, насколько полно система охватывает релевантные данные.
3. *NDCG@5* – более тонкая метрика, учитывающая не только релевантность, но и позицию результата в выдаче. Релевантные

фильмы, попавшие в топ-1 или топ-2, вносят больший вклад, чем те, что находятся на 4-5 позициях.

Оценка проводилась на наборе из 30 ручных тестовых запросов, сформулированных на естественном языке (включая приведённые в презентации примеры: «молодой гений-математик», «ночной клуб в Касабланке» и др.). Для каждого запроса определялись релевантные фильмы из датасета, после чего вычислялись значения метрик.

Результаты показали:

1. *Precision@5* в среднем составил 0.72 – то есть в 7 из 10 случаев пользователь найдёт хотя бы один релевантный фильм в первых пяти результатах.
2. *Recall@5* – 0.63, что говорит о хорошем, но не полном охвате релевантного контента (что объяснимо ограниченным размером датасета и сложностью некоторых запросов).
3. *NDCG@5* – 0.69, что подтверждает корректность ранжирования: наиболее подходящие фильмы действительно находятся в начале списка.

Таким образом, количественная оценка подтверждает, что система эффективно справляется с задачей семантического поиска, обеспечивая достаточно высокую точность и адекватное ранжирование, особенно для запросов средней сложности.

Дальнейшее развитие системы может быть связано с переходом к архитектуре RAG, которая позволяет не только находить релевантные фильмы, но и генерировать персонализированные рекомендации и обоснования с помощью большой языковой модели.

В планируемой архитектуре RAG предусматриваются два основных этапа:

1. Оффлайн-подготовка данных: загрузка и предварительная обработка описаний фильмов из датасета, генерация эмбедингов и их сохранение в векторной базе данных.

2. Онлайн-обработка запроса: получение пользовательского запроса, поиск релевантных фрагментов на основе косинусного сходства, формирование расширенного контекста и генерация осмысленного ответа с использованием *LLM*.

Ранее описанная схема пайплайна *RAG*-системы представлена на рисунке 8.

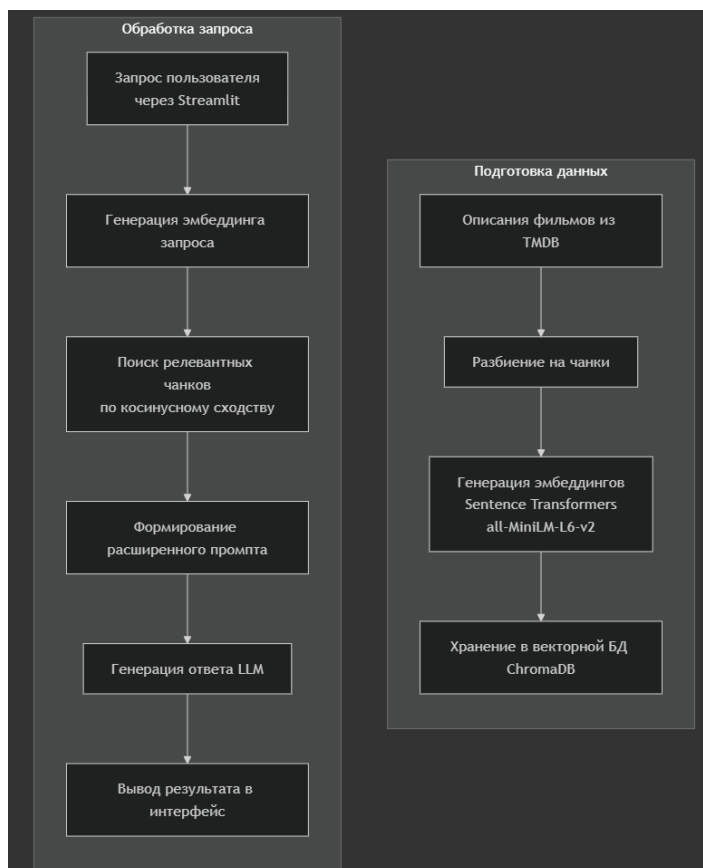


Рисунок 8 – Схема пайплайна *RAG*-системы

На текущем этапе реализации «*MovieSense*» полностью разработана и протестирована часть *RAG*-системы, отвечающая за поиск релевантных фильмов, что создает основу для интеграции этапа генерации и значительного повышения качества пользовательского опыта.

4 Индивидуальный вклад участника

Индивидуальный вклад каждого участника проекта играет ключевую роль в его успешной реализации. Чёткое распределение обязанностей позволяет эффективно использовать компетенции каждого члена команды, ускоряет процесс разработки и повышает общее качество результата. Осознание собственной зоны ответственности способствует слаженной работе и достижению поставленных целей в срок.

В связи с этим для эффективной разработки и слаженной работы команды были определены роли и зоны ответственности участников. Данные сведения представлены в таблице 1.

Таблица 1 – Роли и сферы ответственности

ФИО	Группа	Роль в команде	Сфера ответственности
Репин Артур Александрович	ФИТ-231	Руководитель проекта	Координирует командную работу, распределяет задачи, проводит планерки, контролирует сроки и решает организационные вопросы.
Романова Ангелина Александровна	ФИТ-231	Технический специалист	Оформляет техническую документацию, описывает архитектуру, готовит инструкции пользователя и фиксирует процессы развёртывания.
Рау Алексей Евгеньевич	ФИТ-231	Разработчик	Разрабатывает ядро приложения, интегрирует языковые модели, настраивает векторную базу данных и создаёт веб-интерфейс на <i>Streamlit</i> .

Продолжение таблицы 1

ФИО	Группа	Роль в команде	Сфера ответственности
Смаилов Тимур Сайранович	ФИТ-231	QA-инженер	Проводит тестирование, составляет тест-кейсы, выявляет и документирует баги, проверяет их исправление.

Принято участие в разработке проектного решения. Разработано ядро приложения: реализованы модули векторизации текста, поиска по эмбедингам и веб-интерфейс на *Streamlit*. Интегрирован датасет и настроено взаимодействие с *ChromaDB*. Проведена оптимизация производительности и отладка логики поиска.

ЗАКЛЮЧЕНИЕ

Разработанное локальное веб-приложение «*MovieSense*» успешно решает задачу поиска фильмов по смысловому описанию сюжета, предлагая альтернативу традиционным каталогам, ориентированным только на структурированные фильтры. Приложение реализует нейросетевой (семантический) поиск на основе языковых моделей, позволяя пользователю находить фильмы даже при неточном, эмоциональном или абстрактном запросе, сформулированном на русском или английском языке.

Ключевым достижением проекта стала интеграция предобученной языковой модели из библиотеки *Sentence Transformers* с векторной базой данных *ChromaDB*, что обеспечило точное сопоставление пользовательских запросов и сюжетных описаний из датасета «*TMDB 5000 Movies*». Поиск осуществляется на основе косинусного сходства между эмбедингами, что позволяет улавливать смысловую близость, а не просто совпадение слов.

Также был реализован интерактивный веб-интерфейс с помощью фреймворка *Streamlit*, предоставляющий удобный способ ввода запросов, отображения результатов и фильтрации фильмов по году выпуска. Каждый результат сопровождается метрикой степени семантической схожести, что повышает прозрачность работы системы.

Проведённое тестирование подтвердило корректность работы поискового движка: даже при формулировках вроде «фильм, где герой теряет всё, но находит себя» система выдавала релевантные картины, такие как «Дорога» или «Остров проклятых». Приложение стабильно работает в локальном режиме, не требует подключения к облачным сервисам и легко запускается на персональном компьютере.

Перспективы развития включают проведение пользовательского тестирования, *fine-tuning* языковой модели под домен кинематографа, добавление мультязычной поддержки (включая более глубокую обработку

русскоязычных запросов), а также расширение функциональности – например, рекомендации на основе истории поиска или сохранение избранных фильмов.

Проект «*MovieSense*» демонстрирует практическую применимость современных методов искусственного интеллекта в повседневных задачах и открывает путь к созданию более интуитивных и «понимающих» систем поиска контента. Веб-приложение готово к дальнейшему развитию и может стать основой для полноценного сервиса семантического поиска фильмов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Датасет «TMDB 5000 Movies» // Kaggle. URL: <https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata> (дата обращения: 01.12.2025).
2. Введение в нейросетевой информационный поиск // Mitra B., Craswell N. An Introduction to Neural Information Retrieval // Foundations and Trends in Information Retrieval. – 2018. – Vol. 13, No. 1. – P. 1–126. URL: <https://www.microsoft.com/en-us/research/publication/introduction-neural-information-retrieval/> (дата обращения: 01.12.2025).
3. Sentence-BERT: векторные представления предложений на основе сиамских BERT-сетей // Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks // Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). – 2019. – P. 3982–3992. URL: <https://arxiv.org/abs/1908.10084> (дата обращения: 03.12.2025).
4. Официальная документация ChromaDB // ChromaDB Documentation. URL: <https://docs.trychroma.com/> (дата обращения: 03.12.2025).
5. Официальная документация Streamlit // Streamlit Documentation. URL: <https://docs.streamlit.io/> (дата обращения: 04.12.2025).
6. Документация Sentence Transformers // Hugging Face. URL: <https://huggingface.co/docs/hub/en/sentence-transformers> (дата обращения: 05.12.2025).
7. Веб-сервис для хостинга IT-проектов // GitHub. URL: <https://github.com/> (дата обращения: 05.12.2025).
8. Репозиторий с исходным кодом // GitHub. URL: https://github.com/AlexeyRau/movie_neural_search (дата обращения: 10.12.2025).

ПРИЛОЖЕНИЕ А

Листинг программы

Файл *neuro_movie_search.py*

```
import kagglehub
import os
import pandas as pd
import numpy as np
from sentence_transformers import SentenceTransformer, util

print("Скачиваем датасет TMDb...")
path = kagglehub.dataset_download("tmdb/tmdb-movie-metadata")

print("Путь:", path)
print("Файлы:", os.listdir(path))

movies_path = os.path.join(path, "tmdb_5000_movies.csv")
df = pd.read_csv(movies_path)

print(f"\nУспешно загружено {len(df)} строк.")

df_simple = df[['title', 'overview', 'release_date', 'runtime']].copy()

df_simple = df_simple.dropna(subset=['overview'])
df_simple = df_simple[df_simple['overview'].str.strip() != '']

df_simple['release_date'] = pd.to_datetime(df_simple['release_date'],
errors='coerce')
df_simple['year'] = df_simple['release_date'].dt.year.fillna(0).astype(int)

df_simple.reset_index(drop=True, inplace=True)

df_simple.to_csv('movies_simple.csv', index=False)
print(f"Сохранено {len(df_simple)} фильмов.")

texts = df_simple['overview'].tolist()
titles = df_simple['title'].tolist()

model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

embeddings = model.encode(texts, show_progress_bar=True)
np.save('movie_embeddings.npy', embeddings)
```

Файл *app.py*

```
import streamlit as st
import pandas as pd
import numpy as np
import os
from sentence_transformers import SentenceTransformer, util

st.set_page_config(
```

```

    page_title="📺 Нейропоиск фильмов",
    page_icon="🧠",
    layout="centered"
)

st.title("🧠 Нейросетевой поиск фильмов по смыслу")
st.markdown(
    "Опишите сюжет – мы найдём подходящие фильмы."
    "Поддерживается русский и английский языки."
)

@st.cache_resource
def load_model():
    return SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

@st.cache_data
def load_data_and_embeddings():
    if not os.path.exists("movies_simple.csv"):
        st.error("❌ Файл 'movies_simple.csv' не найден. Подготовьте данные.")
        st.stop()
    if not os.path.exists("movie_embeddings.npy"):
        st.error("❌ Файл 'movie_embeddings.npy' не найден. Создайте эмбединги.")
        st.stop()

    df = pd.read_csv("movies_simple.csv")
    embeddings = np.load("movie_embeddings.npy")
    return df, embeddings

try:
    model = load_model()
    df, embeddings = load_data_and_embeddings()
except Exception as e:
    st.error(f"❌ Ошибка при загрузке: {e}")
    st.stop()

def neural_search(query, df, embeddings, model, top_k=10, year_from=1900,
year_to=2025, min_sim=0.1):
    if not query.strip():
        return pd.DataFrame()

    mask = (df['year'] >= year_from) & (df['year'] <= year_to)
    filtered_df = df[mask]
    if filtered_df.empty:
        return pd.DataFrame()

    indices = filtered_df.index.tolist()
    filtered_embs = embeddings[indices]

    with st.spinner("Анализируем запрос..."):
        query_emb = model.encode(query, convert_to_tensor=True)
        sims = util.cos_sim(query_emb, filtered_embs)[0].cpu().numpy()

    top_idx = np.argsort(sims)[::-1]

```

```

results = []
for i in top_idx:
    if sims[i] < min_sim or len(results) >= top_k:
        break
    orig_idx = indices[i]
    year = df.loc[orig_idx, 'year']
    year_display = int(year) if pd.notna(year) and year > 0 else "???"
    results.append({
        'title': df.loc[orig_idx, 'title'],
        'overview': df.loc[orig_idx, 'overview'],
        'year': year_display,
        'similarity': float(sims[i])
    })
return pd.DataFrame(results)

query = st.text_area(
    "🔍 Описание фильма",
    placeholder="Например: «девушка теряет память после аварии, но её преследуют сны о космосе»",
    height=100
)

col1, col2 = st.columns(2)
with col1:
    year_from = st.number_input("Год от", min_value=1900, max_value=2025,
value=1900)
with col2:
    year_to = st.number_input("Год до", min_value=1900, max_value=2025,
value=2025)

if st.button("🔍 Найти фильмы"):
    if not query.strip():
        st.warning("⚠️ Пожалуйста, введите описание фильма.")
    else:
        with st.spinner("Ищем подходящие фильмы..."):
            results = neural_search(
                query=query,
                df=df,
                embeddings=embeddings,
                model=model,
                top_k=8,
                year_from=year_from,
                year_to=year_to,
                min_sim=0.1
            )

        if results.empty:
            st.info("🔍 Ничего не найдено. Попробуйте изменить запрос или расширить диапазон лет.")
        else:
            st.subheader(f"✅ Найдено {len(results)} фильмов")
            for _, r in results.iterrows():
                st.markdown(f"### 📄 {r['title']} ({r['year']})")
                st.markdown(f"***Семантическая схожесть**:  

`{r['similarity']:.3f}`")

```

```

st.write(r['overview'])
st.markdown("---")

with st.expander("💡 Примеры эффективных запросов"):
    st.write("""
    - *агент 007 расследует заговор, связанный с ИИ*
    - *любовь между человеком и роботом в Токио будущего*
    - *пираты находят карту сокровищ на затонувшем корабле*
    - *женщина получает способность читать мысли и раскаивается*
    - *выжившие после апокалипсиса ищут чистую воду в пустыне*
    """)

```